

Universidade do Minho
Licenciatura em Engenharia Informática — Redes de Computadores 2025/2026

Trabalho Prático N.º 2

Packet Sniffer e Analisador de Tráfego

Grupo PL91

Gonçalo Monteiro — a110727
Afonso Barbosa — a111074
José Gomes — a110367

Data de entrega

6 de maio de 2026

Índice

1. Introdução	3
2. Arquitetura do Sniffer	3
3. Protocolos Identificados	4
3.1. ARP — Address Resolution Protocol	4
3.2. ICMP — Internet Control Message Protocol	4
3.3. ICMPv6... Equivalente ao ICMP para redes IPv6. O sniffer identifica ICMPv6EchoRequest e ICMPv6EchoReply com os respectivos identificadores e números de sequência, tratando-os como camada separada do IPv6.	5
3.4. DNS — Domain Name System	5
3.5. DHCP — Dynamic Host Configuration Protocol	5
3.6. HTTP — HyperText Transfer Protocol	6
3.7. TCP — Transmission Control Protocol	6
3.8. UDP — User Datagram Protocol	7
3.9. IPv6	7
3.10. IEEE 802.11 (Wi-Fi)	7
4. Filtros e Parâmetros de Execução	7
4.1. Modo Interativo	8
4.1.1. Filtro BPF	10
5. Logging e Persistência de Dados	11
6. Topologia no Emulador CORE	11
6.1. Descrição da Topologia	11
6.2. Tráfego Gerado e Protocolos Capturados	12
7. Testes Efetuados	12
7.1. Teste 1 — ICMP no CORE (ping)	12
7.2. Teste 2 — TCP na Interface Real	13
7.3. Teste 3 — Filtro por Protocolo	14
7.4. Teste 4 — ARP no CORE	14
7.5. Teste 5 — DNS na Interface Real	14
7.6. Teste 6 — DHCP na Interface Real	15
7.7. Teste 7 — HTTP no CORE	16
8. Análise na Interface Real	17
9. Limitações	17
10. Opcionais	17
10.1. Modo Interativo	18
10.2. Estatísticas Periódicas	18
10.3. Hierarquia de Protocolos	18
11. Conclusão	19

1. Introdução

Este relatório descreve o desenvolvimento de um *packet sniffer* no âmbito da unidade curricular de Redes de Computadores. O objetivo do trabalho foi implementar uma ferramenta capaz de capturar, identificar e analisar pacotes de rede em tempo real, de forma análoga ao Wireshark, aplicando os conhecimentos teóricos adquiridos nas aulas.

A ferramenta foi desenvolvida em **Python** com recurso à biblioteca **Scapy** e testada em dois ambientes distintos: uma rede emulada no CORE e a interface de rede real do computador. Suporta múltiplos protocolos, filtros de captura a vários níveis, modo *live* com cor por protocolo e persistência de dados em ficheiro (TXT, CSV e JSON).

2. Arquitetura do Sniffer

O sniffer está organizado em quatro componentes principais, cada um responsável por uma fase distinta do processamento de pacotes:

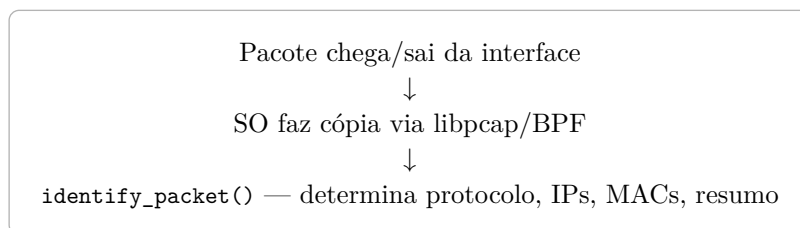
Captura de pacotes — Realizada através da função `sniff` da Scapy, que internamente utiliza a `libpcap`. Esta biblioteca opera ao nível do kernel através do mecanismo BPF (Berkeley Packet Filter), fazendo uma cópia de cada pacote que atravessa a interface selecionada sem interferir com o tráfego original. A captura é completamente passiva.

Identificação e parsing — A função `identify_packet` analisa cada pacote e determina o protocolo, os endereços relevantes e um resumo do conteúdo. A identificação segue uma hierarquia de camadas: Ethernet → ARP/DHCP → IPv4/IPv6 → ICMP/DNS/TCP/UDP → HTTP/802.11.

Filtragem — Suportada a dois níveis: ao nível do kernel via expressões BPF (passadas diretamente ao Scapy/libpcap) e ao nível da aplicação por protocolo, endereço IP ou endereço MAC.

Saída e logging — O sniffer pode operar em modo *live* (impressão na consola com cores por protocolo), em modo *log* (escrita em ficheiro TXT, CSV e/ou JSON) ou em ambos simultaneamente. As estatísticas de captura podem ser apresentadas no final ou periodicamente durante a captura.

O fluxo de execução é o seguinte:



↓
Filtros por protocolo / IP / MAC
↓
Saída *live* (consola) e/ou *log* (TXT / CSV / JSON)

3. Protocolos Identificados

3.1. ARP — Address Resolution Protocol

O ARP opera na camada de ligação e é responsável por resolver endereços IP em endereços MAC dentro da mesma rede local. O sniffer distingue dois tipos de mensagens: o **ARP Request** (difusão a perguntar “quem tem este IP?”) e o **ARP Reply** (resposta unicast com o MAC correspondente). A identificação baseia-se no campo `op` do cabeçalho ARP: `op=1` para Request, `op=2` para Reply.

Diagrama ARP

Host A → (broadcast) : ARP Request — “Quem tem 10.0.0.22?”
Host B → Host A : ARP Reply — “10.0.0.22 está em 00:00:00:aa:00:02”

ARP Request: Quem tem 10.0.0.22? Diz a 10.0.0.20
ARP Reply: 10.0.0.22 está em 00:00:00:aa:00:02

3.2. ICMP — Internet Control Message Protocol

O ICMP é um protocolo de controlo da camada de rede, utilizado para diagnóstico e reporte de erros. O sniffer identifica os seguintes tipos com base nos campos `type` e `code`:

- Echo Request (`type=8`) e Echo Reply (`type=0`) — gerados pelo comando `ping`
- Destination Unreachable (`type=3`) — host ou porta inacessível
- Time Exceeded (`type=11`) — TTL esgotado, base do `traceroute`

Diagrama ICMP (ping)

PC1 → sniffer : Echo Request (`type=8`, `id=27072`, `seq=96`)
sniffer → PC1 : Echo Reply (`type=0`, `id=27072`, `seq=96`)

```
ICMP Echo Request id=27072 seq=96
ICMP Echo Reply id=27072 seq=96
ICMP Dest Unreachable code=3
ICMP Time Exceeded (TTL=0)
```

3.3. ICMPv6... Equivalente ao ICMP para redes IPv6. O sniffer identifica ICMPv6EchoRequest e ICMPv6EchoReply com os respectivos identificadores e números de sequência, tratando-os como camada separada do IPv6.

3.4. DNS — Domain Name System

O DNS traduz nomes de domínio em endereços IP, operando sobre UDP na porta 53. O sniffer distingue **DNS Query** (campo qr=0) de **DNS Response** (campo qr=1), extraindo o nome pedido via `dns.qd.qname` e os endereços devolvidos iterando pelos registros `dns.an`.

Diagrama DNS

```
Cliente → Servidor (porta 53) : DNS Query — qr=0,
                                     “mobile.events.data.microsoft.com.”
Servidor → Cliente           : DNS Response — qr=1,
                                     “mobile.events.data.trafficmanager.net.”
```

```
DNS Query: mobile.events.data.microsoft.com.
DNS Response: mobile.events.data.trafficmanager.net.
```

3.5. DHCP — Dynamic Host Configuration Protocol

O DHCP atribui configurações de rede automaticamente, operando sobre UDP (portas 67/68). O sniffer identifica o tipo de mensagem através da opção `message-type` do cabeçalho DHCP, cobrindo as quatro fases do processo DORA:

Diagrama DHCP (processo DORA)

```
Cliente → (broadcast 255.255.255.255) : DHCP Discover — src=0.0.0.0
Servidor → Cliente                     : DHCP Offer   — IP proposto,
                                     GW, DNS
Cliente → (broadcast 255.255.255.255) : DHCP Request — aceita o IP
                                     oferecido
Servidor → Cliente                     : DHCP ACK     — confirma,
                                     lease ativo
```

3.6. HTTP — HyperText Transfer Protocol

O HTTP é o protocolo da camada de aplicação para transferência de páginas web, operando sobre TCP na porta 80. O sniffer detecta este protocolo verificando se a porta de origem ou destino é 80/8080 e se existe uma camada `HTTPRequest` ou `HTTPResponse` da Scapy, extraindo método, host e caminho (pedidos) ou código de estado e razão (respostas).

Diagrama HTTP

Cliente → Servidor (porta 80) : HTTP GET /
Servidor → Cliente : HTTP Response 200 OK + dados

```
HTTP GET 10.0.0.21/  
HTTP Response 200 OK
```

3.7. TCP — Transmission Control Protocol

O TCP é o protocolo de transporte orientado à ligação. O sniffer analisa as flags de cada segmento através da função `_tcp_flags()`, que faz AND bit a bit com os valores de cada flag para gerar uma string legível:

- **SYN** — pedido de estabelecimento de ligação
- **SYN|ACK** — aceitação pelo servidor
- **ACK** — confirmação (three-way handshake completo)
- **PSH|ACK** — transferência de dados
- **FIN|ACK** — encerramento ordenado
- **RST** — encerramento abrupto

Diagrama TCP — Three-Way Handshake e encerramento

Cliente → Servidor : SYN (seq=x)
Servidor → Cliente : SYN|ACK (seq=y, ack=x+1)
Cliente → Servidor : ACK (ack=y+1)
(ligação estabelecida)

Cliente → Servidor : PSH|ACK (dados)
Servidor → Cliente : PSH|ACK (dados)

Cliente → Servidor : FIN|ACK
Servidor → Cliente : FIN|ACK
Cliente → Servidor : ACK
(ligação encerrada)

```
TCP 10.0.0.22:55702 → 10.0.0.21:80 [SYN] seq=3499540024
TCP 10.0.0.21:80 → 10.0.0.22:55702 [SYN|ACK] seq=3783742804
TCP 10.0.0.22:55702 → 10.0.0.21:80 [ACK] seq=3499540025
```

3.8. UDP — User Datagram Protocol

O UDP é o protocolo de transporte sem ligação. O sniffer extrai as portas de origem e destino e o tamanho do datagrama através do campo `udp.len`. É o protocolo base de DNS e DHCP.

3.9. IPv6

Versão 6 do protocolo IP, com endereços de 128 bits. O sniffer trata IPv6 separadamente do IPv4, verificando a presença da camada IPv6 e ausência de IP. O protocolo de camada superior é identificado através do campo `next-header`.

3.10. IEEE 802.11 (Wi-Fi)

Protocolo da camada de ligação para redes sem fio. O sniffer identifica **Beacon frames** (anúncios periódicos de redes Wi-Fi com o SSID, obtido via `network_stats()`) e **Probe Requests** (enviados por dispositivos à procura de redes conhecidas). A captura requer que a interface esteja em modo *monitor*.

4. Filtros e Parâmetros de Execução

O sniffer pode ser executado de forma interativa (sem argumentos) ou via linha de comandos com os seguintes parâmetros:

Parâmetro	Exemplo	Descrição
<code>-i / --iface</code>	<code>-i eth0</code>	Interface de rede a escutar
<code>-n / --count</code>	<code>-n 100</code>	Nº de pacotes a capturar (0 = infinito)
<code>-f / --proto</code>	<code>-f ICMP</code>	Filtro por protocolo
<code>--ip</code>	<code>--ip 10.0.0.20</code>	Filtro por endereço IP
<code>--mac</code>	<code>--mac aa:bb:cc:dd:ee:ff</code>	Filtro por endereço MAC
<code>--bpf</code>	<code>--bpf "tcp port 80"</code>	Filtro BPF ao nível do kernel
<code>--live / --no-live</code>	<code>--no-live</code>	Ativar/desativar saída na consola
<code>--log-txt</code>	<code>--log-txt cap.txt</code>	Guardar log em ficheiro TXT
<code>--log-csv</code>	<code>--log-csv cap.csv</code>	Guardar log em ficheiro CSV
<code>--log-json</code>	<code>--log-json cap.json</code>	Guardar log em ficheiro JSON
<code>--stats-interval</code>	<code>--stats-interval 5</code>	Estatísticas a cada N segundos
<code>--list-ifaces</code>	<code>--list-ifaces</code>	Listar interfaces disponíveis

Tabela 1: Parâmetros de execução do sniffer.

4.1. Modo Interativo

Quando executado sem argumentos, o sniffer entra em modo de configuração interativa, guiando o utilizador passo a passo pela seleção de interface, protocolo, número de pacotes e filtros opcionais.

As figuras seguintes ilustram o menu em funcionamento no nó *sniffer* do CORE....

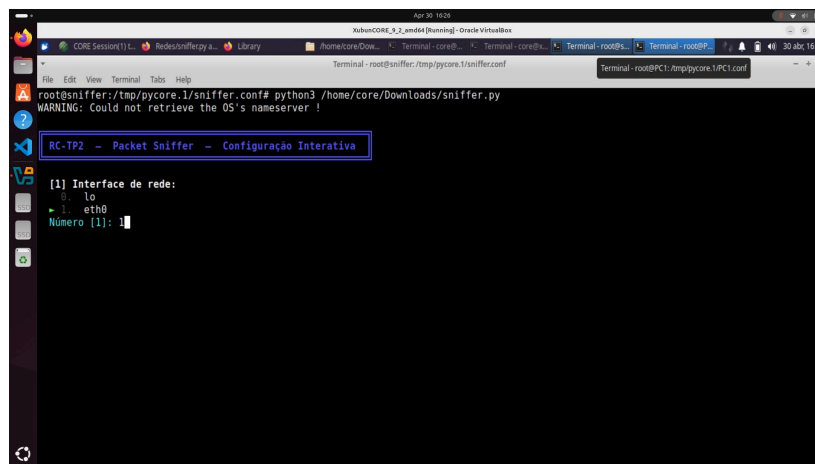


Figura 1: Passo 1: seleção da interface de rede. O utilizador escolhe *eth0* (opção 1).

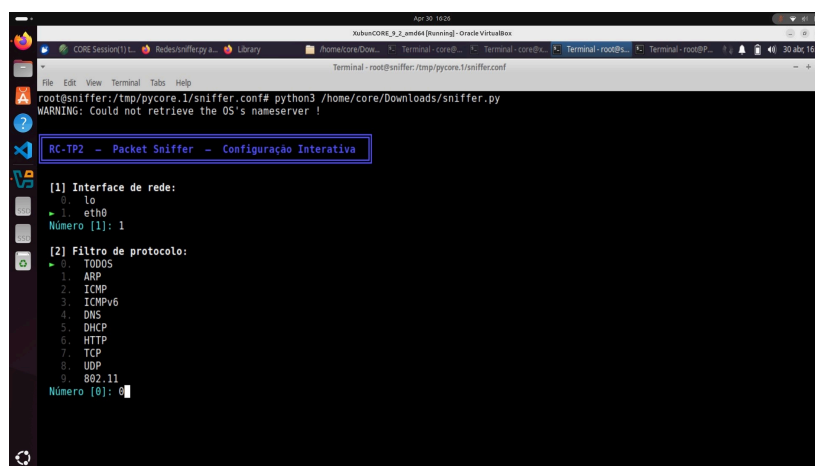


Figura 2: Passo 2: seleção do filtro de protocolo. Opção 0 (*TODOS*) captura todos os protocolos.

Exemplos de expressões BPF válidas: `tcp, udp port 53, host 10.0.0.20, tcp port 80`.

5. Logging e Persistência de Dados

O sniffer suporta três formatos de log, que podem estar ativos simultaneamente. Todos os ficheiros são escritos e descarregados (`flush`) após cada pacote, garantindo que os dados não se perdem em caso de interrupção abrupta. O encerramento é feito no handler de `SIGINT` (Ctrl+C).

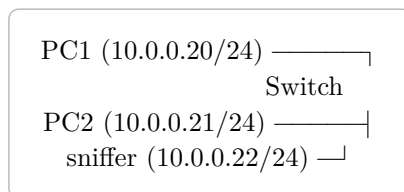
Formato	Campos	Uso recomendado
TXT	<code>timestamp, #n, proto, src, dst, size, summary</code>	Leitura humana, auditoria rápida
CSV	<code>n, timestamp, iface, proto, src, dst, mac_src, mac_dst, size, summary</code>	Importação em Excel / pandas
JSON Lines	Mesmos campos que o CSV, um objeto por linha	Processamento programático

Tabela 2: Formatos de log suportados e respetivos campos.

6. Topologia no Emulador CORE

6.1. Descrição da Topologia

A topologia utilizada no emulador CORE é composta por três nós ligados a um switch central, todos na mesma rede `10.0.0.0/24`:



O nó **sniffer** foi criado especificamente para correr a ferramenta desenvolvida. O **PC1** e o **PC2** geram o tráfego. Como todos estão ligados ao mesmo switch, o sniffer recebe os pacotes destinados a si ou originados por si; para capturar tráfego entre outros nós foi ativado o modo promíscuo com `ip link set eth0 promisc on`.

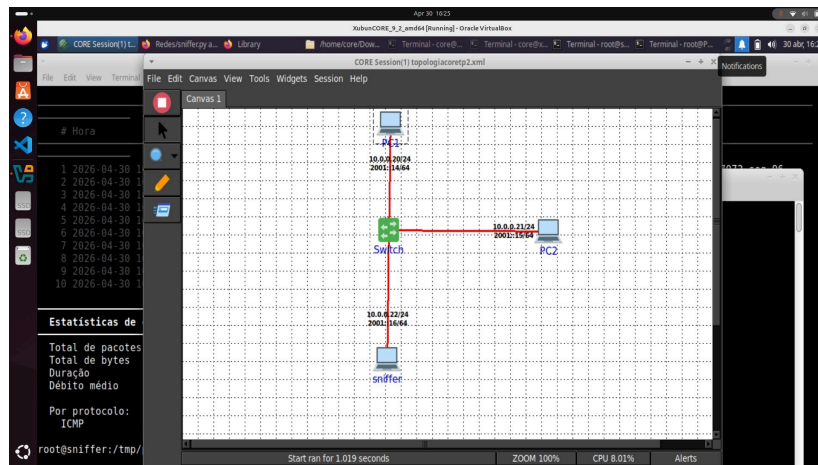


Figura 5: Topologia no emulador CORE: PC1 (10.0.0.20), PC2 (10.0.0.21) e sniffer (10.0.0.22) ligados a um switch central.

6.2. Tráfego Gerado e Protocolos Capturados

Para validar o funcionamento em ambiente controlado, o PC1 executou ping 10.0.0.22, gerando tráfego ICMP capturado pelo sniffer. A captura registrou pares de Echo Request / Echo Reply com incremento sequencial do campo seq, confirmando a correta identificação do protocolo.

7. Testes Efetuados

7.1. Teste 1 — ICMP no CORE (ping)

Campo	Detalhe
Objetivo	Validar a captura e identificação de tráfego ICMP em ambiente controlado.
Procedimento	No nó PC1, executou-se ping 10.0.0.22. O sniffer foi configurado para capturar 10 pacotes na interface eth0.
Resultado	10 pacotes ICMP capturados, alternando entre Echo Request (PC1 → sniffer) e Echo Reply (sniffer → PC1), com incremento correto de seq=96 a seq=100.
Estatísticas	10 pacotes 980 bytes duração 4.7 s débito médio 0.20 KB/s

Tabela 3: Teste 1: ICMP no CORE.

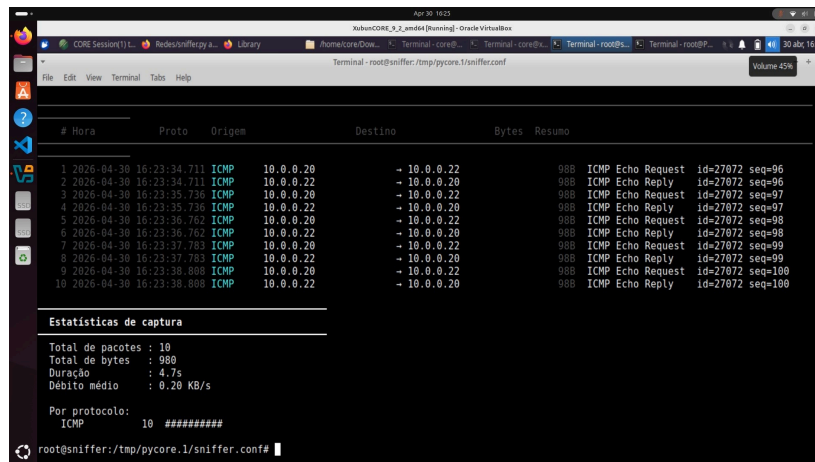


Figura 6: Saída do sniffer: 10 pacotes ICMP capturados com Echo Request e Echo Reply, estatísticas finais e histograma por protocolo.

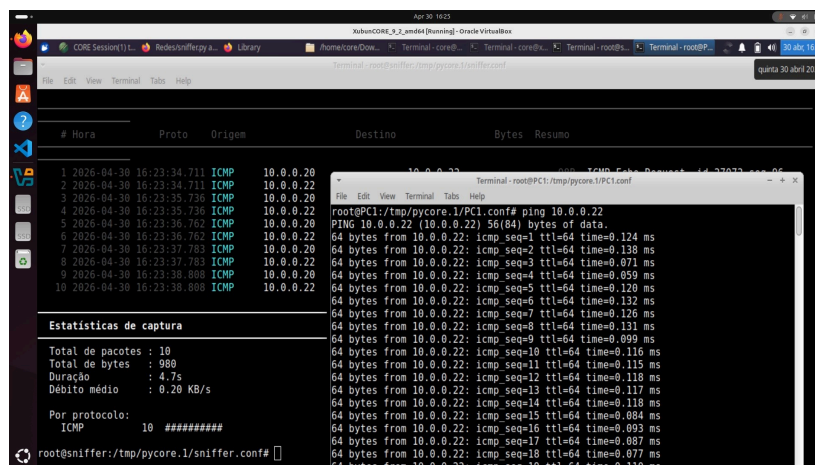


Figura 7: Vista simultânea: sniffer (esquerda) e terminal do nó PC1 a executar ping 10.0.0.22 (direita).

7.2. Teste 2 — TCP na Interface Real

Objetivo: Validar a identificação do ciclo de vida de uma ligação TCP.

Procedimento: Captura de 10 pacotes na interface en0 do computador real, sem filtros adicionais.

Resultado: Foi capturada uma ligação TCP completa à porta 993 (IMAP sobre SSL), evidenciando o three-way handshake (SYN → SYN|ACK → ACK), transferência de dados (PSH|ACK) e encerramento (FIN|ACK e RST).

7.3. Teste 3 — Filtro por Protocolo

Objetivo: Validar o filtro por protocolo ao nível da aplicação.

Procedimento: Execução do sniffer com `--proto ICMP` e geração de tráfego misto (ping + navegação web).

Resultado: Apenas pacotes ICMP foram apresentados na saída, confirmando o funcionamento correto do filtro. Os pacotes TCP e DNS gerados pelo browser foram descartados antes da impressão.

7.4. Teste 4 — ARP no CORE

Objetivo: Validar a captura e identificação de mensagens ARP.

Procedimento: No nó PC1, executou-se `ip neigh flush all` para limpar a cache ARP, seguido de `ping 10.0.0.22 -c 1`. O sniffer capturou o tráfego na interface `eth0` com filtro `-f ARP`.

Resultado: Foram capturados um ARP Request (PC1 a perguntar pelo MAC de 10.0.0.22) e um ARP Reply (sniffer a responder com 00:00:00:aa:00:02), confirmando a resolução de endereços antes do envio dos pacotes ICMP.

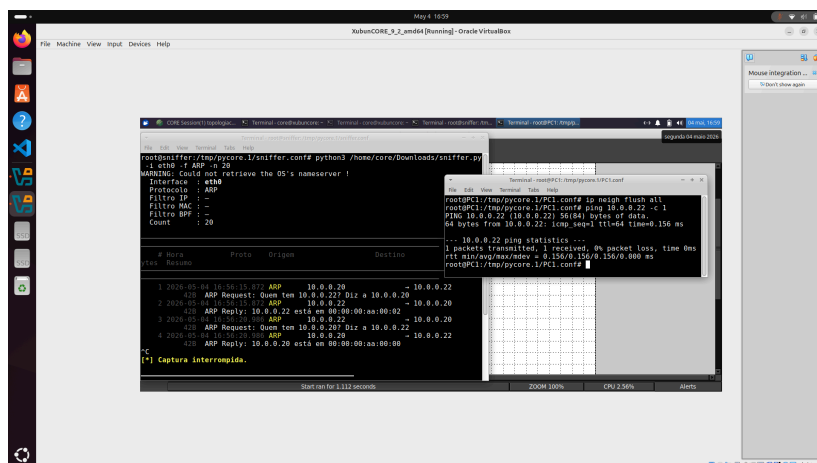


Figura 8: Teste 4: ARP Request e ARP Reply capturados no CORE. PC1 (10.0.0.20) resolve o MAC do sniffer (10.0.0.22).

7.5. Teste 5 — DNS na Interface Real

Objetivo: Validar a captura e identificação de tráfego DNS.

Procedimento: Na interface `wlp0s20f3` do computador real, executou-se o sniffer com `--bpf "udp port 53"`. O sistema operativo e as aplicações em execução geraram automaticamente consultas DNS.

Resultado: Foram capturados pares de DNS Query (qr=0) e DNS Response (qr=1). O sniffer identificou corretamente o nome pedido (mobile.events.data.microsoft.com.) e a resposta devolvida pelo router (192.168.1.254), confirmando a identificação do protocolo DNS sobre UDP porta 53.

```

# sniffer.py
219 def des_info(pkt, src, dst, eth_src, eth_dst):
220     return "DNS", src, dst, summary, eth_src, eth_dst
221
222
223 def _des_info(pkt, src, dst, eth_src, eth_dst):
224     dhcp_opts = [opt for opt in pkt[DHCP].options if isinstance(opt, tuple)]
225     msg_type = dhcp_opts[0][1] if dhcp_opts else 0
226     types = {1: "Discover", 2: "Offer", 3: "Request", 4: "Decline",
227             5: "Ack", 6: "Nack", 7: "Release", 8: "Inform"}
228     t = types.get(msg_type, "Unknown")
229     return t, dhcp_opts
230
231
232 # Sniffer
233 def sniff(interface, callback, filter):
234     sock = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(0))
235     sock.bind((interface, 0))
236     filter_str = filter
237     while True:
238         data, addr = sock.recvfrom(4096)
239         if filter_str:
240             if not filter_str.startswith("ethernet II"):
241                 continue
242             if not filter_str.startswith("IP"):
243                 continue
244             if not filter_str.startswith("UDP"):
245                 continue
246             if not filter_str.startswith("DNS"):
247                 continue
248         pkt = scapy.Packet(data)
249         src, dst, eth_src, eth_dst = des_info(pkt)
250         t, dhcp_opts = _des_info(pkt, src, dst, eth_src, eth_dst)
251         callback(pkt, src, dst, eth_src, eth_dst, t, dhcp_opts)
252
253
254 # Teste
255 if __name__ == '__main__':
256     interface = 'wlp0s20f3'
257     filter_str = 'udp port 53'
258     sniff(interface, callback, filter_str)
259
260
261 # Estatísticas de captura
262 total_packets = 0
263 total_bytes = 0
264 duration = 0
265 data_received = 0
266
267 for packet in packets:
268     total_packets += 1
269     total_bytes += len(packet)
270     duration += 1
271     data_received += len(packet[0])
272
273 print("Total de pacotes: %d" % total_packets)
274 print("Total de bytes: %d" % total_bytes)
275 print("Duração: %d s" % duration)
276 print("Dados recebidos: %d KB" % (data_received / 1024))
277
278 # Por protocolo
279 protocols = {}
280 for packet in packets:
281     protocol = packet[0]
282     protocols[protocol] = protocols.get(protocol, 0) + 1
283
284 for protocol, count in protocols.items():
285     print("%s: %d" % (protocol, count))
286
287 # Gomes-Aspire-A515-57 - Desktop/Documents/Reds

```

Figura 9: Teste 5: DNS Query e DNS Response capturados na interface real (wlp0s20f3). Resolução de mobile.events.data.microsoft.com.

7.6. Teste 6 — DHCP na Interface Real

Objetivo: Validar a captura das quatro fases do processo DORA.... **Procedimento:** Na interface wlp0s20f3, executou-se o sniffer com --bpf "udp port 67 or udp port 68". Em paralelo, forçou-se a renovação do endereço IP com sudo dhclient -r wlp0s20f3 && sudo dhclient wlp0s20f3.

Resultado: Foram capturadas as cinco mensagens do processo — Release, Discover, Offer, Request e ACK — confirmando o funcionamento completo do protocolo DHCP. O IP atribuído foi 192.168.1.96 ao host gomes-Aspire-A515-57.

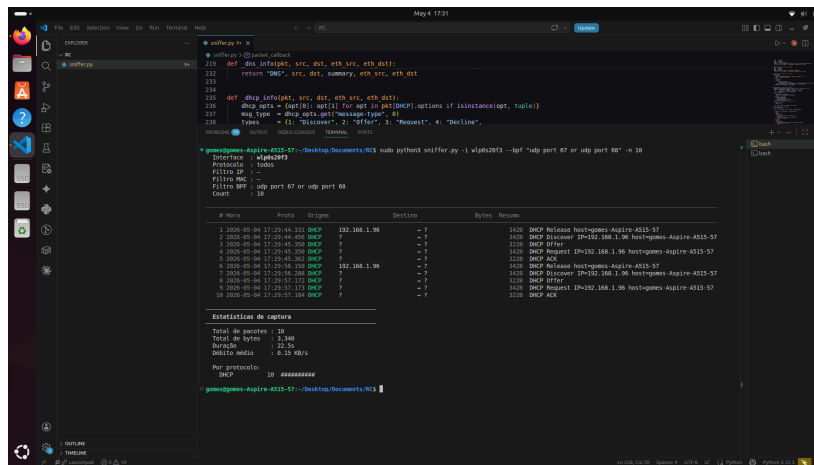


Figura 10: Teste 6: Processo DORA completo capturado na interface real — Release, Discover, Offer, Request e ACK.

7.7. Teste 7 — HTTP no CORE

Objetivo: Validar a identificação de pedidos e respostas HTTP.

Procedimento: No CORE, o nó PC2 executou um servidor HTTP com `python3 -m http.server 80`. O sniffer iniciou a captura na interface `eth0` e, em segundo terminal, executou `curl http://10.0.0.21/` diretamente a partir do nó sniffer, garantindo que o tráfego passasse pela sua interface.

Resultado: Foram capturados o three-way handshake TCP (`SYN` → `SYN|ACK` → `ACK`), seguido de um HTTP GET e uma HTTP Response 200 OK, confirmando a correta identificação do protocolo HTTP sobre TCP porta 80.

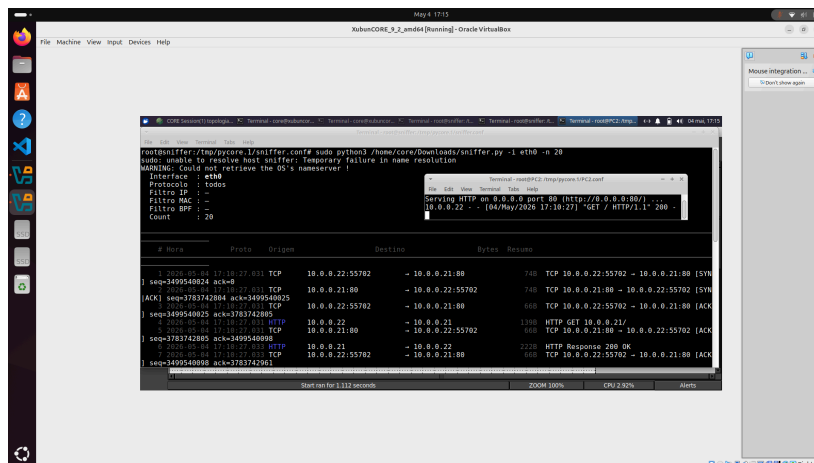


Figura 11: Teste 7: HTTP GET e Response 200 OK capturados no CORE, precedidos pelo three-way handshake TCP (`SYN`, `SYN|ACK`, `ACK`).

8. Análise na Interface Real

O sniffer foi executado na interface `wlp0s20f3` (Wi-Fi) do computador do aluno com permissões de root. Neste ambiente foi possível observar tráfego gerado pelo sistema operativo e pelas aplicações em execução.

O protocolo predominante foi TCP, nomeadamente ligações a servidores de email (porta 993 — IMAPS) e a servidores web (porta 443 — HTTPS). Foram também observados pacotes DNS, gerados automaticamente para resolução de nomes, e tráfego DHCP aquando da renovação do endereço IP.

Em comparação com o ambiente CORE, a interface real apresenta um volume de tráfego significativamente superior e mais diversificado, com múltiplas ligações simultâneas a diferentes servidores. O protocolo 802.11 não foi capturado porque a interface estava em modo *managed* (não *monitor*), o que impede a captura de frames Wi-Fi de outros dispositivos.

9. Limitações

- **Fragmentação IP** — O sniffer não suporta *reassembly* de pacotes IP fragmentados. Os fragmentos são capturados individualmente mas não são reagrupados para análise do payload completo.
- **Tráfego cifrado** — Tráfego cifrado (TLS/HTTPS, IMAPS) é capturado ao nível TCP mas o conteúdo da camada de aplicação não é acessível.
- **Modo monitor Wi-Fi** — A captura de frames 802.11 requer que a interface Wi-Fi esteja em *monitor mode*, o que nem sempre é possível sem configuração adicional do SO.
- **Switch no CORE** — O switch do CORE encaminha pacotes apenas para o destinatário correto. Para capturar tráfego entre outros nós foi necessário ativar o modo promíscuo (`ip link set eth0 promisc on`) ou originar o tráfego diretamente no nó sniffer.
- **Retransmissões TCP** — O sniffer não deteta retransmissões TCP nem analisa a janela de congestionamento.
- **Desempenho sem BPF** — Em redes com elevado débito, a utilização de filtros exclusivamente ao nível da aplicação (sem BPF) pode introduzir atraso no processamento.

10. Opcionais

Para além dos requisitos base, foram implementadas três funcionalidades adicionais.

10.1. Modo Interativo

Quando executado sem argumentos, o sniffer apresenta um menu de configuração passo a passo (`interactive_setup()`), guiando o utilizador pela seleção de interface, protocolo, número de pacotes, filtros opcionais e ficheiros de log. Esta funcionalidade melhora significativamente a usabilidade, especialmente em contexto de demonstração, eliminando a necessidade de memorizar os parâmetros da linha de comandos.

10.2. Estatísticas Periódicas

O parâmetro `--stats-interval N` ativa uma *thread* dedicada (`_stats_loop`) que imprime automaticamente as estatísticas de captura (total de pacotes, bytes, débito médio e histograma por protocolo) a cada N segundos, sem interromper a captura. Isto permite monitorizar a evolução do tráfego em tempo real durante capturas longas.

10.3. Hierarquia de Protocolos

À semelhança do Wireshark, o sniffer apresenta no final de cada captura (ou periodicamente via `--stats-interval`) uma vista de hierarquia de protocolos. Para cada protocolo identificado são mostrados o número absoluto de pacotes e bytes capturados, bem como a respetiva percentagem em relação ao total. A tabela respeita a estrutura de encapsulamento das camadas (Frame → Ethernet → ARP / IPv4 / IPv6 → TCP / UDP → HTTPS / DNS), permitindo uma leitura imediata da distribuição do tráfego capturado.

A Figura 12 ilustra o resultado obtido numa captura de 21 pacotes (2 499 bytes) efetuada na interface real. Verifica-se que a totalidade do tráfego era Ethernet encapsulado maioritariamente em IPv4 (90,5 % dos pacotes), sendo o TCP/HTTPS o protocolo dominante com 57,1 % dos pacotes e 39,2 % dos bytes. O único pacote ARP representa 4,8 % dos pacotes mas apenas 2,4 % dos bytes (60 bytes), consistente com o cabeçalho ARP de dimensão fixa. O pacote IPv6/UDP/DNS destaca-se pelo tamanho superior (20,4 % dos bytes), evidenciando uma resposta DNS com *payload* mais extenso.

Hierarquia de Protocolos				
Protocolo	% Pacotes	Pacotes	% Bytes	Bytes
Frame	100.0%	21	100.0%	2,499
Ethernet	100.0%	21	100.0%	2,499
ARP	4.8%	1	2.4%	60
IPv4	90.5%	19	77.2%	1,930
TCP	57.1%	12	39.2%	980
HTTPS	57.1%	12	39.2%	980
UDP	33.3%	7	38.0%	950
DNS	4.8%	1	19.6%	489
IPv6	4.8%	1	20.4%	509
UDP	4.8%	1	20.4%	509
DNS	4.8%	1	20.4%	509

Figura 12: Hierarquia de protocolos gerada pelo sniffer — captura de 21 pacotes (2 499 bytes) na interface real. Estrutura análoga à vista de hierarquia do Wireshark.

A Tabela 4 detalha os valores numéricos exibidos na figura anterior.

Protocolo	% Pacotes	Pacotes	% Bytes	Bytes
Frame	100,0 %	21	100,0 %	2 499
Ethernet	100,0 %	21	100,0 %	2 499
ARP	4,8 %	1	2,4 %	60
IPv4	90,5 %	19	77,2 %	1 930
TCP	57,1 %	12	39,2 %	980
HTTPS	57,1 %	12	39,2 %	980
UDP	33,3 %	7	38,0 %	950
DNS	4,8 %	1	19,6 %	489
IPv6	4,8 %	1	20,4 %	509
UDP	4,8 %	1	20,4 %	509
DNS	4,8 %	1	20,4 %	509

Tabela 4: Valores da hierarquia de protocolos — captura de 21 pacotes na interface real.

11. Conclusão

O trabalho permitiu consolidar os conhecimentos teóricos sobre protocolos de rede através da sua observação prática. O sniffer desenvolvido cumpre os requisitos estabelecidos: captura passiva de pacotes, identificação de múltiplos protocolos, filtros a vários níveis, modos *live* e *log*, e estatísticas de captura.

A utilização do emulador CORE permitiu validar o funcionamento em ambiente controlado antes da transposição para a interface real. A análise na interface real evidenciou a complexidade do tráfego numa rede doméstica, com múltiplos protocolos e ligações simultâneas.

O principal desafio técnico foi a correta identificação da hierarquia de protocolos — distinguir DHCP de UDP genérico, ou HTTP de TCP genérico — resolvido através de uma função de identificação que percorre as camadas do pacote de forma ordenada.